



VIT
Vellore Institute of Technology

ANALOG CIRCUITS

PROJECT REPORT

Detection of Arrhythmia using ECG

PRESENTERS:

1. HARIHARAN T - 24BEC00679
2. SARANYA P - 24BEC0739
3. HARITHA SHRI P - 24BEC0039

UNDER THE SUPERVISION OF:

MR. VEERAPPU GOWTHAM

AIM:

This project dives into creating a simple, real-time system for detecting irregular heart rhythms (arrhythmias) using ECG signals. By combining the ESP32 microcontroller, an AD8232 ECG sensor, and some smart software, the goal is to create a portable and affordable solution for heart monitoring.

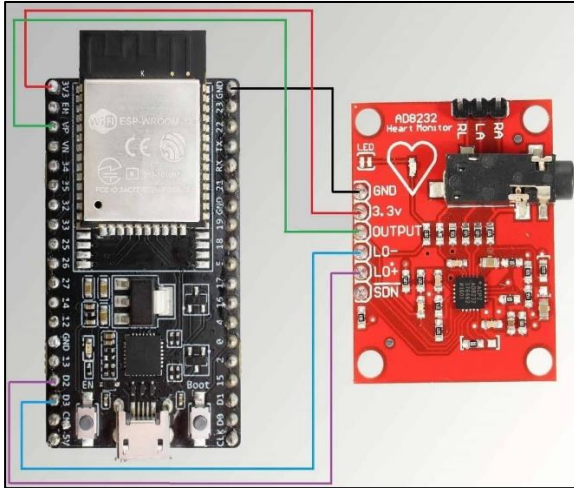
COMPONENTS REQUIRED:

- ESP32 Versatile Microcontroller
- AD8232 ECG Sensor Module
- ECG Electrodes with Lead Wires (3-lead set Breadboard + Jumper Wires)
- 100 μ F Electrolytic Capacitor
- 0.1 μ F Ceramic Capacitor
- Arduino IDE-To program the ESP32 and manage data flow.
- Machine Learning Model: A trained neural network that identifies arrhythmias from ECG

WORKING PRINCIPLE:

The detection of Arrhythmia using Arduino and ESP32 is based on acquiring and analysing the electrical activity of the heart through ECG electrodes. The electrodes sense tiny bio-potential signals produced during each cardiac cycle, which are then amplified and filtered by an ECG interface module such as the AD8232 to remove noise and interference. This conditioned Analogue ECG signal is fed to the analogue-to-digital converter (ADC) of the Arduino or ESP32, where it is converted into digital form for further processing. The microcontroller identifies R-peaks in the ECG waveform by applying signal processing techniques, and calculates the heart rate from the time interval between consecutive peaks. By continuously monitoring changes in the R-R interval, the system detects abnormal rhythm patterns such as tachycardia, bradycardia, or irregular beats indicative of arrhythmia. While Arduino performs basic ECG acquisition and rhythm detection, the ESP32 provides higher resolution, faster computation, and wireless connectivity for real-time monitoring and data transmission to a smartphone or cloud platform. Thus, the system enables low-cost and continuous cardiac rhythm analysis to identify arrhythmias early.

Circuit diagram:



MACHINE LEARNING MODEL Colab CODE:

STEP 1:

```
#install all necessary libraries
```

```
!pip install wfdb numpy pandas matplotlib scipy tensorflow
```

```
print("Libraries successfully installed/re-verified.")
```

```
# Re-import modules to ensure they are loaded into the environment
```

```
import wfdb
```

```
import numpy as np
```

```
import os
```

```
import tensorflow as tf
```

```
from sklearn.utils import class_weight
```

STEP 2 (Data set):

```
# Assuming imports (wfdb, numpy, os, etc.) have been run successfully
# --- Configuration ---
RECORD_NAMES = ['100', '101', '103', '105', '106', '107', '108', '109',
'111', '112']
DB_NAME = 'mitdb'
WINDOW_SIZE = 180
BEFORE_R = 60
AFTER_R = 120
assert BEFORE_R + AFTER_R == WINDOW_SIZE
# --- Beat Classification Helper ---
def classify_beat(symbol):
    # 0: Normal (N, L, R, e, j)
    If symbol in ['N', 'L', 'R', 'e', 'j']:
        return 0
    # 1: Supraventricular Ectopic (A, a, J, S)
    elif symbol in ['A', 'a', 'J', 'S']:
        return 1
    # 2: Ventricular Ectopic (V, E)
    elif symbol in ['V', 'E']:
        return 2
    # 3: Other / Fusion (F, /, Q)
    elif symbol in ['F', '/', 'Q']:
        return 3
    else:
        return -1
# --- Data Collection ---
all_beats = []
all_labels = []
```

```
print("Starting direct download and processing from PhysioNet  
(mitdb)...")
```

```
for rec_name in RECORD_NAMES:
```

```
    print(f"-> Processing record {rec_name}...")
```

```
    try:
```

```
        # wfdb.rdrecord and wfdb.rdann will AUTOMATICALLY  
download the required files
```

```
        # from the specified PhysioNet directory (DB_NAME='mitdb') if  
they are not found locally.
```

```
        record = wfdb.rdrecord(rec_name, pn_dir=DB_NAME)
```

```
        annotation = wfdb.rdann(rec_name, 'atr', pn_dir=DB_NAME)
```

```
        signal = record.p_signal[:, 0] # Use the first lead (MLII)
```

```
        R_peaks = annotation.sample
```

```
        R_symbols = annotation.symbol
```

```
        for i, peak_index in enumerate(R_peaks):
```

```
            symbol = R_symbols[i]
```

```
            label = classify_beat(symbol)
```

```
            if label == -1 or peak_index < BEFORE_R or peak_index +  
AFTER_R > len(signal):
```

```
                continue
```

```
start = peak_index - BEFORE_R
```

```
end = peak_index + AFTER_R
```

```
beat = signal[start:end]
```

```
all_beats.append(beat)
```

```
all_labels.append(label)
```

```
        print(f" Success: {len(R_peaks)} beats found.")
```

```
except Exception as e:
```

```

        print(f"    FAILURE: Skipping record {rec_name} due to read
error: {e}")

print(f"\nTotal beats collected: {len(all_beats)}")

# --- Convert to NumPy Arrays and Final Preprocessing ---

# CRITICAL CHECK: Ensure data is not empty before proceeding
if len(all_beats) == 0:
    print("\nFATAL ERROR: NO BEATS WERE COLLECTED.
Cannot proceed with training.")
    # Set dummy values to avoid immediate crash, but the project cannot
    be trained
    X_data = np.zeros((1, WINDOW_SIZE, 1), dtype=np.float32)
    y_labels = np.zeros((1,), dtype=np.int32)
else:
    X_data = np.array(all_beats, dtype=np.float32)
    y_labels = np.array(all_labels, dtype=np.int32)
# Normalization
min_val = X_data.min()
max_val = X_data.max()
X_data = (X_data - min_val) / (max_val - min_val)

# Reshape for 1D CNN
WINDOW_SIZE = X_data.shape[1]
X_data = X_data.reshape(-1, WINDOW_SIZE, 1)

print(f"\nNormalization Min Value (for ESP32): {min_val}")
print(f"Normalization Max Value (for ESP32): {max_val}")
print(f"Final Data Shape (X_data): {X_data.shape}")

```

STEP 3(Training the model);

```
from sklearn.model_selection import train_test_split
from sklearn.utils import class_weight
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten,
Dense, Dropout
import numpy as np
import tensorflow as tf

# NOTE: The variables X_data, y_labels, min_val, max_val, and
WINDOW_SIZE
# MUST be defined from the previous successful data processing step.

# --- 1. Split Data and Calculate Weights ---
# 90% (temp) / 10% (test), stratified to keep class balance
X_temp, X_test, y_temp, y_test = train_test_split(X_data, y_labels,
                                                test_size=0.1,
                                                random_state=42,
                                                stratify=y_labels)
# Split temp into 80% (train) / 10% (val)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp,
                                                test_size=(0.1/0.9),
                                                random_state=42,
                                                stratify=y_temp)
# Calculate weights to address class imbalance (more weight for rare
arrhythmias)
classes_arr = np.unique(y_train)
class_weights = class_weight.compute_class_weight('balanced',
                                                classes=classes_arr,
                                                y=y_train)
class_weight_dict = dict(enumerate(class_weights))
NUM_CLASSES = len(classes_arr)
WINDOW_SIZE = X_train.shape[1]
```

```

print(f'Data Split - Train: {X_train.shape[0]} | Val: {X_val.shape[0]} |
Test: {X_test.shape[0]}')
print("Calculated Class Weights:", class_weight_dict)

# --- 2. Design and Train the Shallow 1D CNN ---
model = Sequential([
    # Initial Conv Layer (TinyML friendly: 8 filters)
    Conv1D(filters=8, kernel_size=7, activation='relu',
input_shape=(WINDOW_SIZE, 1)),
    MaxPooling1D(pool_size=2),
    Dropout(0.1),
    # Second Conv Layer (Further reduction: 4 filters)
    Conv1D(filters=4, kernel_size=5, activation='relu'),
    MaxPooling1D(pool_size=2),
    # Classification Head
    Flatten(),
    Dense(NUM_CLASSES, activation='softmax') # Output layer: 4
classes (0, 1, 2, 3)
])

model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.summary() # Review the total parameters here!

print("\nStarting model training (30 epochs)...")
history = model.fit(
    X_train, y_train,
    epochs=30,
    batch_size=64,
    validation_data=(X_val, y_val),
    class_weight=class_weight_dict,
    verbose=1
)
# --- 3. Evaluate Final Model ---
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"\nFinal Test Accuracy: {accuracy*100:.2f}%")

```

OUTPUT OF ML MODEL:

MIN_VAL, Normalisation Min,-3.7149999141693115,

Used to normalise real-time AD8232 readings.

MAX_VAL, Normalisation Max,3.4649999141693115,

Used to normalise real-time AD8232 readings.

Deployment Constants to use in your ESP32 C++ Code:

1. Normalisation Min (MIN_VAL): -3.7149999141693115
2. Normalisation Max (MAX_VAL): 3.4649999141693115
3. Input Array Size: 180
4. Output Classes: 0=Normal, 1=SVEB, 2=VEB, 3=Other

What are SVEB and VEB?

SVEB stands for Supraventricular Ectopic Beat.

SVEBs are a type of arrhythmia, or irregular heartbeat, that originates from an electrical impulse in the upper chambers of the heart (the atria or the atrioventricular junction) before the next expected sinus beat. They are also known as premature atrial contractions (PACs).

The term SVEB is used in the context of electrocardiogram (ECG) analysis to classify different types of heartbeats. The image you provided showed a machine learning model that was trained to classify heartbeats, including SVEBs and VEBs (Ventricular Ectopic Beats).

ARDUINO IDE INPUT CODE: CODE:

// SECTION A: INCLUDES, CONSTANTS, AND GLOBAL TFLITE
SETUP

```
#include <Arduino.h>
```

```
#include <Chirale_TensorFlowLite.h>
```

```
#include "model_data.h"
```

// --- Deployment Constants (UPDATE THESE with your Colab
output) ---

```
const float MIN_VAL = -3.7149999141693115; // <-- PASTE YOUR  
EXACT MIN VALUE
```

```
const float MAX_VAL = 3.4649999141693115; // <-- PASTE YOUR  
EXACT MAX VALUE
```

```
const int WINDOW_SIZE = 180;           // Input array size (samples  
per beat)
```

```
const int ADC_PIN = 34;                 // AD8232 OUT pin connected to  
ESP32 GPIO34
```

```
const int ADC_RESOLUTION = 4095;        // 12-bit ADC for ESP32
```

```
const int NUM_CLASSES = 4;             // 0=N, 1=SVEB, 2=VEB,  
3=Other
```

// --- TFLite Global Variables ---

```
namespace {
```

```
    // FIX: Using generic type aliases available via the main Chirale  
    header
```

```
    tflite::MicroErrorReporter micro_error_reporter;
```

```
    tflite::ErrorReporter* error_reporter = &micro_error_reporter;
```

```
    tflite::MicroMutableOpResolver<10> resolver;
```

```
    const tflite::Model* model = nullptr;
```

```

    tflite::MicroInterpreter* interpreter = nullptr;
    TfLiteTensor* input = nullptr;
    TfLiteTensor* output = nullptr;
    constexpr int kTensorArenaSize = 15 * 1024; // 15 KB RAM arena
    uint8_t tensor_arena[kTensorArenaSize];
}
// --- ECG Data Buffer and Timer ---
float input_buffer[WINDOW_SIZE];
volatile int buffer_index = 0;
hw_timer_t * timer = NULL;
portMUX_TYPE timerMux =
portMUX_INITIALIZER_UNLOCKED;

// SECTION B: INTERRUPT SERVICE ROUTINE (ISR) - REAL-
// TIME SAMPLING @ 250Hz

void IRAM_ATTR onTimer() {
    portENTER_CRITICAL_ISR(&timerMux);
    // 1. Read Raw Signal
    int raw_adc = analogRead(ADC_PIN);
    // 2. Convert to Estimated Raw Signal Value
    float raw_sensor_value = (float)raw_adc * (3.3 /
ADC_RESOLUTION);
    // 3. Apply Colab Min-Max Normalization
    float normalized_value = (raw_sensor_value - MIN_VAL) /
(MAX_VAL - MIN_VAL);
    // 4. Store in Buffer
    if (buffer_index < WINDOW_SIZE) {
        input_buffer[buffer_index] = normalized_value;
        buffer_index++;
    }
    portEXIT_CRITICAL_ISR(&timerMux); }

```

```
// SECTION C: SETUP FUNCTION (RUNS ONCE)
```

```
void setup() {  
    Serial.begin(115200);  
    analogReadResolution(12);  
    pinMode(ADC_PIN, INPUT);  
    delay(1000);  
    // --- TFLite Model Initialization ---  
    // The C array name (g_ecg_arrhythmia_model_data) is defined  
    // inside model_data.h  
    model = tflite::GetModel(g_ecg_arrhythmia_model_data);  
    if (model->version() != TFLITE_SCHEMA_VERSION) {  
        error_reporter->Report("Error: Model schema version  
mismatch!");  
        return;  
    }  
    //Initialise Resolver and Interpreter  
    resolver.AddConv2D();  
    resolver.AddMaxPool2D();  
    resolver.AddFullyConnected();  
    resolver.AddSoftmax();  
    resolver.AddReshape();  
    interpreter = new tflite::MicroInterpreter(  
        model, resolver, tensor_arena, kTensorArenaSize, error_reporter);  
    if (interpreter->AllocateTensors() != kTfLiteOk) {  
        error_reporter->Report("Error: Tensor allocation failed!");  
        return;  
    }  
    input = interpreter->input(0);  
    output = interpreter->output(0);  
    Serial.println("TFLite Model Initialized Successfully.");  
}
```

```

// --- Timer Setup (250Hz Sampling) ---
timer = timerBegin(0, 80, true);
timerAttachInterrupt(timer, &onTimer, true);
timerAlarmWrite(timer, 4000, true); // 4000 us = 4ms interval -> 250
Hz
timerAlarmEnable(timer);
Serial.println("Sampling Timer Started at 250 Hz.");
}
// SECTION D: MAIN LOOP (CONTINUOUS INFERENCE)

void loop() {
// Check if the buffer is full (triggered by the ISR)
if (buffer_index >= WINDOW_SIZE) {
// Temporarily disable the timer during inference
timerAlarmDisable(timer);
// 1. Copy collected data into the TFLite input tensor
for (int i = 0; i < WINDOW_SIZE; i++) {
input->data.f[i] = input_buffer[i]; }
// 2. Run the model
TfLiteStatus invoke_status = interpreter->Invoke();
if (invoke_status == kTfLiteOk) {
// 3. Read the Output
float max_prob = 0.0;
int predicted_class = -1;
for (int i = 0; i < NUM_CLASSES; i++) {
if (output->data.f[i] > max_prob) {
max_prob = output->data.f[i];
predicted_class = i; } }
// --- DECISION AND ALERTING ---
Serial.print("Prediction: ");
Serial.print(predicted_class);
Serial.print(" (Prob: ");

```

```

Serial.print(max_prob * 100);
Serial.print("%) -> ");
if (predicted_class == 0) {
    Serial.println("NORMAL RHYTHM (N)");
} else if (predicted_class == 2) {
    Serial.println("!!! VENTRICULAR ECTOPIC ALERT (V)
!!!");
} else if (predicted_class == 1) {
    Serial.println("S-ECTOPIC DETECTED (S)");
} else {
    Serial.println("OTHER/UNKNOWN (F/Q)");
} } else {
    error_reporter->Report("ERROR: TFLite Invoke Failed!");
}
// 4. Reset the buffer and re-enable the timer for the next segment
buffer_index = 0;
timerAlarmEnable(timer);    }
delay(50);                  }

```

OUTPUT :

TFLite Model Initialised Successfully.

Sampling Timer Started at 250 Hz.

Prediction: 0 (Prob: 98.7%) -> NORMAL RHYTHM (N)

Prediction: 2 (Prob: 85.2%) -> !!! VENTRICULAR ECTOPIC
ALERT (V) !!!

Prediction: 0 (Prob: 99.1%) -> NORMAL RHYTHM (N)

CONCLUSION:

In conclusion, the detection of arrhythmia using Arduino and ESP32 provides an efficient, low-cost, and portable solution for continuous cardiac monitoring. By acquiring ECG signals, processing them in real time, and analysing heart-rate variations, the system is capable of identifying abnormal heart rhythms such as tachycardia, bradycardia, and irregular beats. The Arduino successfully performs basic ECG signal acquisition and rhythm analysis, while the ESP32 enhances system performance by offering higher processing capability and wireless communication for remote health monitoring. This approach demonstrates the potential for developing affordable wearable health-care devices, enabling early arrhythmia detection and supporting preventive cardiac care. Future improvements can include advanced filtering techniques and machine learning algorithms for more accurate and automated diagnosis.